

FROM CHANDRA

• BEGINNER-FRIENDLY WALK-THROUGH

The AI That **Checks** Its Own Work

A step-by-step build of an agent that drafts, critiques itself, and rewrites — until the answer is actually good.

— SWIPE TO BUILD ONE



Chandra Sekhar
AI Engineer & Trainer



Wait — what's an **AI agent**?

It's not just a chatbot that replies. An **agent** is a tiny program built around an LLM that can:

- 1 Decides what to do next
- 2 Takes an action (call a tool, write code, search)
- 3 Reads its own result
- 4 Keeps going until the goal is met

💡 Think of an agent as **an LLM with a plan, a memory, and a steering wheel.**

The trouble with **one-shot** answers

You prompt. The model replies. Done — even when the reply is shaky, wrong, or just half-right.

❗ **No second pass.**

Whatever rolls off the model first becomes the final answer — even if it's wrong.

❗ **No quality bar.**

There's no internal voice asking 'is this actually good enough to send?'

❗ **No retry logic.**

Hallucinations, sloppy tone, missing facts — they all reach your user untouched.

Enter the critique loop

What if the model wrote a draft, paused, and then read it back as a strict editor — over and over until it was actually good?

THE PATTERN

Same model. Two roles. **One smarter output.**



It's the **writer-and-editor** relationship — except both seats are filled by your LLM.

Meet LangGraph

Regular LangChain runs in a straight line. LangGraph lets you draw your agent as a **flowchart** — with loops, branches, and conditions.

- **Nodes** — small Python functions — one per step
- **Edges** — the lines connecting them
- **Conditional edges** — branches that depend on the state
- **State** — a shared dictionary every node can read & update
- **Cycles** — loops that keep running until you say stop



The shape of our agent

Just two nodes and one decision. The critic decides whether to ship it or send it back.

NODE 1

write_draft()

LLM writes the first attempt



NODE 2

critique_draft()

LLM grades its own work

SHIP IT ✓

→ END node

REVISE ↺

→ back to write_draft()

Design the shared memory

Every node reads and writes the same dictionary. Lock its shape down with a **TypedDict** so nothing surprises you later.

```
from typing import TypedDict

class AgentState(TypedDict):
    topic: str # what the user asked
    draft: str # the current attempt
    notes: str # the critic's feedback
    passed: bool # good enough to ship?
    attempts: int # loop counter
```

Write the two tiny functions

Each node receives the state, calls the LLM, and returns the keys it wants to update. That's it.

```
def write_draft(state):  
    prompt = f"Write about: {state['topic']}"  
    return {"draft": llm.invoke(prompt)}  
  
def critique_draft(state):  
    prompt = f"""Grade this draft.  
{state['draft']}  
Reply PASS or REVISE: <why>"""  
    reply = llm.invoke(prompt)  
    passed = reply.startswith("PASS")  
    return {"passed": passed, "notes": reply}
```

Wire it into a graph

Register the nodes, draw the edges, and let the **conditional edge** decide whether to loop or exit.

```
from langgraph.graph import StateGraph

g = StateGraph(AgentState)
g.add_node("write", write_draft)
g.add_node("critique", critique_draft)
g.add_edge("write", "critique")
g.add_conditional_edges("critique",
    lambda s: "end" if s["passed"] else "write"
)
g.set_entry_point("write")
agent = g.compile()
```

Don't forget the **safety net**

A picky critic with no patience can loop forever. Always give your agent an **exit door**.

- 🛡️ Track an **attempts** counter inside the state
- 🛡️ Force-exit at **max_attempts** (3 is usually enough)
- 🛡️ Log every draft and critique for **debugging & replay**
- 🛡️ Plug in **LangSmith** to watch the loop run live



Where this pattern pays off

Reach for it whenever the **first answer can't be the last answer**.

LONG-FORM WRITING

Blog posts, cold emails, product copy — auto-edited until the tone lands.

CODE GENERATION

Write a function, then critique it for bugs, edge cases, and security holes.

SUMMARIZATION

Catch missing key points before the summary ever reaches the reader.

GROUNDED Q&A

Make sure each answer is actually backed by the retrieved sources.

THAT'S A WRAP

Your AI just learned to **double-check** itself.

Hit follow for weekly walk-throughs on Python, LangGraph, AI agents, and RAG — straightforward, no hype.

- Follow Chandra for more AI content

@CHANDRA